

Janiya Green

Brandon Walker

Github : xxklusivenana

Github : simplyexpired

Repo Url: <https://github.com/xxklusivenana/inclass1v3-janiyagreen-brandonwalker>

Critical Thinking Questions

1. Widget Tree. Draw or describe the widget tree of your In-Class 01b app, at least four levels deep. If a future employer asked you to add a fifth tab, which single node in your tree would you need to change first, and why does that node "own" the number of tabs?

Scaffold –AppBar-TabBar-Tabs 1,2,3, and 4

Scaffold TabBarView-Tab-Column-Text-Buttons-images-and others

Scaffold,BottomAppBar,Text

If i wanted to add a fifth tab i would change the tab-controller length from 4 to 5. I would change this first because the tab-controller keeps track of how many tabs there are

2. Stateless vs. Stateful. Pick one widget in your app that is stateless and one that is stateful. Explain, in plain language, what would break (or simply become unnecessary complexity) if you swapped which type each one was. Stateless vs. Stateful
 - The Text widget is stateless and only displays information manually coded in there. This is only applicable in myApp. Inside Tab two, there is a stateful widget called TextField. It changes constantly while typing and keeps track of user input.
 - It wouldn't make any sense to make the text widget stateful because it doesn't need to remember anything; unnecessary complexity. Meanwhile, switching the TextField would break its usability. It wouldn't be able to handle changing user input which is its whole purpose.
3. Controllers & Lifecycle. Imagine you shipped this app to the App Store without calling `_tabController.dispose()`. Describe, step by step, what would happen in

memory over hours of real-world use, and why a code reviewer at a real company would flag this immediately.

If I didn't use `_tabController.dispose()`, the controller could stay in memory even when I don't need it anymore. Over time this could use more memory and make the app slower and it could also cause it to crash. A code reviewer would point this out because you should clean your own things that aren't being used

If you deleted only the `setState()` call but kept the line updating `tabIndex.value`, would the visible tab still change when tapped? Explain why, tying your answer back to the Declarative UI idea above. Code Deep-Dive

4. Declarative UI. Compare the declarative approach used in Flutter (`setState()` → rebuild) to an imperative approach you may have seen elsewhere (e.g., manually changing an HTML element with JavaScript). Which would be easier to maintain in a large team, and why?
 - Flutter uses a declarative approach that reminds me of Java. Manually changing the text on a label with `label.setText("New Text")`, and hardcoding things in is a good example of this.
 - Overall though, I do feel that Flutter's declarative approach would be easier to maintain for a large team; setting everything up in a container. A large dev team focused on the bigger components as a whole, not just individual components. This just makes the code easier to understand, limits unnecessary lines, and helps in debugging which would help greatly if the app were to grow.
5. Team Collaboration. Reflect on setting up your shared GitHub repository with your partner: what part of the workflow (branches, pull requests, merge conflicts, or communication) felt hardest, and what would you do differently on your very first day at a new software job to avoid that friction?
 - The hardest part would be getting into a flow once we dealt with communication.
6. Restoration Layer. The starter code uses a `RestorableInt` `tabIndex` field and `RestorationMixin` instead of relying on `setState()` alone. Describe a real scenario (e.g., the OS killing your app in the background) where this extra layer matters, and what the user would experience if it weren't there.
 - Another scenario could be a contact form where you have to fill out your information or submit an application online. The user's progress needs to be

saved so that they won't just have to redo everything if an error were to occur. Without restoration, progress could be lost and everything would need to be redone; causing aggravation.

7. Listener → setState. In initState(), `_tabController.addListener(() { setState(() { tabIndex.value = _tabController.index; }); })` connects the controller to the rest of the UI. If you deleted only the `setState()` call but kept the line updating `tabIndex.value`, would the visible tab still change when tapped? Explain why, tying your answer back to the Declarative UI idea above.

If I deleted `setState()`, the `tabIndex` could still change, but the rest of the screen might not update like it should. `setState()` lets Flutter know that something changed and the screen needs to update. The tab could still switch because of the `TabController`, but other stuff on the screen might not update.

8. One list, two loops. Both the `TabBar`'s tabs: and the `TabBarView`'s children: are built with a `for (final tab in tabs)` loop over the same tabs list. In your In-Class 01b build, did you keep this loop pattern or write out each tab/child by hand? Which approach is less likely to cause the "tabs[i] and children[i] don't line up" bug the 01b instructions warned about?
 - I kept the loop pattern.
9. Requirements → code. Reopen `inclass01b.html` and pick one required widget you added (`AlertDialog`, `TextField`, `SnackBar`, or `ListView/Card`). Point to the exact layer from the Concepts page's code walkthrough (e.g., "Layer 7 · Declarative build") where that widget lives, and explain why it belongs at that layer and not, say, inside `initState()`.

One widget I added was the `TextField` in Tab 2. It is in Layer 7, which is the Declarative build layer. It goes there because the `TextField` is something you can see and use on the screen. It wouldn't go inside `initState()` because that is more for setting things up when the app first starts.